Amazon Elastic Container Service

Explore how Amazon Elastic Container Service (ECS) streamlines container orchestration in AWS. Understand its architecture, including provisioning, capacity, and controller layers. Learn about task definitions, deployment strategies, health checks, and IAM roles to effectively deploy, manage, and monitor scalable containerized applications on AWS.

Amazon Elastic Container Service (ECS) is a fully managed container orchestration service provided by AWS. It simplifies the deployment and management of containerized applications, allowing us to run Docker containers at scale. ECS integrates seamlessly with other AWS services, providing a flexible and efficient solution for deploying and managing containerized workloads.

How ECS works

ECS builds and deploys scalable containerized applications on AWS without managing servers. Its architecture is divided into three functional layers: provisioning, controller, and capacity management.

Ask

Provisioning

Provisioning consists of the tools used as an interface to manage and deploy our containers and applications in ECS. There are five provisioning tools we can use

to deploy and manage ECS operations:

- AWS Command Line Interface (CLI)
- AWS Software Development Kit (SDK)
- AWS Cloud Developer Kit (CDK)
- AWS Copilot
- AWS Management Console

Capacity

The **capacity** layer provides the compute infrastructure where containers run. ECS supports three types of compute environments:

- **Amazon EC2:** It is used for VM-based, user-managed capacity.
- **AWS Fargate:** It is used for serverless container execution.
- **On-premises servers:** It is used for running workloads via ECS Anywhere.

These resources are organized into an ECS cluster, a logical grouping of capacity used by the controller to place and run tasks. A cluster doesn't provide compute itself, but it defines the scope within which ECS can manage resources. We can configure a cluster to use one or more capacity types (like EC2 and Fargate in the same cluster).

When using EC2 instances, each instance runs the ECS agent, a lightweight process that does the following:

- It registers the instance with the cluster.
- It starts and stops containers based on instructions from the ECS control plane.
- It reports container and instance status back to ECS.

Ask

The ECS agent is preinstalled on ECS-optimized AMIs but can also be installed manually on compatible Linux distributions. For Fargate, the agent is abstracted

away. AWS manages the underlying infrastructure and container lifecycle for Fargate.

Controller

The **controller** layer is responsible for orchestrating tasks and services. It uses definitions and configuration provided by the developer to schedule, launch, and monitor containers.

At the core of this layer is the task definition, which describes how a container should be configured, including image, CPU, memory, environment variables, and networking settings.

- A task is a single instance of a task definition.
- A service ensures a desired number of tasks are always running, automatically replacing failed tasks to maintain availability.

The ECS controller also monitors resource health and performance. It uses data reported by the ECS container agent to make placement and scaling decisions based on resource availability.

Ask

Layers of ECS

ECS services and task definitions

When ECS starts a task, it creates a running instance of that definition. Tasks can run once (like a batch job), on a schedule (with EventBridge), or continuously as part of a service. Rather than acting merely as an abstraction, task definitions and services together form the foundation of how ECS orchestrates and maintains container workloads. This includes the container image, CPU, memory settings, environment variables, IAM roles, logging configuration, port mappings, and more.

Each revision of a task definition captures changes, enabling rollback or versioned deployment across environments, for example, dev, staging, and prod. Here's a sample ECS Fargate task definition for launching an Apache HTTP Server container that serves a static HTML page. It uses the `httpd:2.4` image, logs to CloudWatch, and exposes a port `80` over TCP:

Ask

index.json Javascript (babel-node-es2024)

```
{
   "containerDefinitions": [
      {
         "command": [
            "/bin/sh -c \"echo '<html> <head> <title>Amazon ECS Sampl
         ],
         "entryPoint": [
            "sh",
            "-c"
         ],
         "essential": true,
         "image": "public.ecr.aws/docker/library/httpd:2.4",
         "logConfiguration": {
```

Sample task definition

A service ensures that a specified number of task instances are maintained within a cluster. It handles restart logic, load balancer integration, and rolling updates. ECS services are essential for stateless, highly available microservices. We can also configure service discovery, deployment strategies, and auto scaling policies within a service, making it a powerful mechanism for running production workloads.

Application lifecycle in ECS

To understand the application lifecycle in ECS, consider a scenario where we want to deploy a microservices-based web application on ECS, with the frontend and backend deployed on two different containers for security reasons.

We will configure a task definition that launches two containers, one running the application's frontend and the other hosting the backend. To improve availability and fault tolerance, we deploy this task definition using an ECS service tha maintains two running tasks across different Availability Zones. This setup ensures that if one zone becomes unavailable, the application can still serve users from

Ask

the other. An Application Load Balancer (ALB) routes incoming traffic to healthy tasks. To handle variable workloads without managing servers, we use AWS Fargate as the compute platform, allowing ECS to scale resources automatically as demand changes.

ECS workflow

Deployment strategies and recovery

Reliability and availability in ECS depend not only on launching tasks but also on robust deployment strategies and failure recovery mechanisms. Amazon ECS supports both native and advanced deployment strategies that prioritize safety, observability, and resilience. For simpler workloads, ECS's built-in rolling deployments may be sufficient, while more complex or sensitive applications may benefit from the added control of blue/green deployments using AWS CodeDeploy.

Comparing deployment strategies

The table below compares rolling deployments and blue/green deployments in ECS across key dimensions relevant to developer workflows, observability, and production-grade recovery strategies:

Ask

Feature/Behavior	Rolling Deployment (Default)	Blue/Green Deployment (via CodeDeploy)
Update Mechanism	Gradual replacement of old tasks with new ones	Deploys new tasks in parallel and shifts traffic after validation
Service Downtime	Minimal, based on configured healthy task thresholds	Zero downtime if the traffic shift is successful
Traffic Routing	Handled implicitly by ECS	Requires Application Load Balancer and listener rules
Control Over Rollout Speed	Controlled via `minimumHealthyPercent` and `maximumPercent`	Controlled via CodeDeploy lifecycle hooks and traffic shift settings
Rollback Capability	Manual (revert to previous task definition revision)	Automated rollback if deployment fails
Validation Before Cutover	Not possible mid-deployment	Yes—can validate the green environment before shifting traffic
Integration Requirements	None beyond ECS	Requires ALB, CodeDeploy app, and deployment group
Observability Tools	CloudWatch metrics and logs	Enhanced observability via CodeDeploy hooks, status tracking
Common Use Cases	Simple service updates, low-risk changes	Critical updates, canary testing, or safety-critical production deployments

Health checks and rollbacks

Once a deployment strategy is in place, ensuring task health, quickly identifying issues, and reliably rolling back are key to application stability and developer confidence. ECS integrates health checks and diagnostics into its orchestration logic, helping us monitor and respond to service failures.

Automating health checks

Ask

Health checks serve as the feedback loop for ECS to determine if tasks are ready to serve traffic. There are two primary types of health checks used within ECS:

- Container level health checks are defined directly within the Dockerfile using the `HEALTHCHECK` directive. For instance, a common health check command may be `CMD curl -f http://localhost/health || exit 1`. These checks allow ECS to determine the internal status of the application itself. We can also configure ECS health checks in the `containerDefinitions` section of the task definition JSON using the `healthCheck` parameter. The code example below shows how defining the health checks using Docker and health check section in the task definition JSON looks like:

Doc...

```
# Use an official base image
FROM node:18-alpine

# Set working directory
WORKDIR /usr/src/app

# Copy application files
COPY . .

# Install dependencies
RUN npm install

# Expose the application port
```

Specifying health checks in Docker

  - Once a health check is active, ECS tracks the container's health using one of the following statuses:

Status	Description
`HEALTHY`	The container passed the last health check.
`UNHEALTHY`	The container failed the health check for the specified number of retries.

Ask

`UNKNOWN`	ECS has not received health check results, for example, a container just started.

- If a container becomes `UNHEALTHY` and is part of a service, ECS stops the task and launches a replacement to maintain the desired count. ECS uses this information to manage deployments, rolling updates, and high availability.

> **Note:** ECS does not restart or replace unhealthy tasks automatically unless it is part of a service.

- **Load balancer health checks:** ECS supports load balancer health checks for services integrated with an Application Load Balancer (ALB) or Network Load Balancer (NLB). If a target (ECS task) fails the load balancer's health check, the ALB/NLB stops routing traffic to it. ECS uses this feedback for deployment decisions during blue/green updates and can also trigger task replacements if used in conjunction with ECS service health logic. We can configure these in the target group settings of the load balancer, not in the task definition. Key configuration options include:
  - **Protocol:** HTTP, HTTPS, or TCP
  - **Path:** The path for ALB like `/health`
  - **Port:** The port on which the container listens
  - **Healthy/unhealthy thresholds:** Number of checks required to change health status
  - **Interval and timeout:** Frequency and time limits for checks

A well-architected health check should be lightweight and fast to execute, on a distinct and predictable endpoint like `/health` or `/status`. Configuration parameters like check intervals and thresholds must be carefully tuned to avoid

Ask

"flapping" behavior, situations where transient issues incorrectly signal task failure.

When a task fails a configured health check, ECS automatically terminates the task and launches a replacement based on the existing task definition to maintain the desired count. This orchestration ensures resilience and availability without manual intervention.

Debugging stuck or unhealthy tasks

Tasks may become stuck in various lifecycle states. Identifying these states begins with understanding their meaning and the conditions that lead to them. A task that remains in the `PENDING` state is typically waiting for required resources to become available. This can include insufficient CPU or memory in the cluster or improperly configured networking components such as subnets or security groups, especially relevant when using the `awsvpc` networking mode. Tasks that move into the `STOPPED` state unexpectedly require more specific attention. The illustration below shows the ECS task lifecycle:

Ask

ECS lifecycle

To begin the process of identifying states, do the following:

- Review the `stoppedReason` and `exitCode` from the ECS console or using the AWS CLI. Often, these fields will point to immediate issues such as container crashes, out-of-memory errors, or missing configuration data.
- CloudWatch Logs can be very useful at this stage, providing application-level traces that help pinpoint unhandled exceptions or misconfigured startup scripts.
- Developers should also confirm that all environment variables and sec are correctly set and accessible.

Ask

For example, using the following CLI command provides access to low-level task status details:

main.sh Shell

```
aws ecs describe-tasks \
  --cluster <cluster_name> \
  --tasks <task-id>
```

Get task details

This output reveals current and last-known statuses, stopped reasons, and individual container states.

Performing safe rollbacks

If a newly deployed task definition causes issues in production, ECS offers distinct rollback paths depending on the deployment strategy. In the case of rolling deployments, developers can manually revert the service to a previously known good task definition revision. This can be done through the AWS Management Console or programmatically via the CLI. For instance, executing the following command switches the service back to the specified revision:

Ask

main.sh Shell

```
aws ecs update-service --cluster <cluster_name> \
  --service <service_name>\
```

```
    --task-definition <task_def_name>:3
```

For blue/green deployments managed by AWS CodeDeploy, rollback is automated. If deployment fails based on predefined thresholds or manual approval rejection, CodeDeploy automatically reverts the traffic back to the blue (original) environment. This rollback is facilitated through the deployment history maintained by CodeDeploy, which tracks all previous states and their outcomes. Leveraging this feature provides a safer deployment pipeline, particularly for critical applications.

To make rollbacks effective, teams should adopt disciplined versioning practices, tagging each successful deployment and testing new revisions in preproduction before proceeding to production.

IAM roles and access control

To enable ECS to interact with other AWS services and perform required operations, we use IAM roles. These roles have permission policies attached that define what actions ECS is allowed to perform on specific resources. ECS uses two distinct IAM roles during the container lifecycle:

- **Task execution role:** It grants ECS permissions to pull images from Amazon ECR, push logs to CloudWatch Logs, and fetch parameters or secrets. This role is assumed by the ECS agent. Here's a sample policy for the ECS task

Ask

execution role that grants access to retrieve secrets from AWS Secrets Manager and decrypt them using AWS KMS:

index.json Javascript (babel-node)

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "kms:Decrypt",
                "secretsmanager:GetSecretValue"
            ],
            "Resource": [
                "arn:aws:secretsmanager:<region>:<aws_account_id>:sec
                "arn:aws:kms:<region>:<aws_account_id>:key/key_id"
            ]
```

Sample policy for ECS task execution role

- **Task role:** This role is assigned to the container. It allows an app to interact with other AWS services like S3, DynamoDB, or Secrets Manager. Here's a sample policy for a task role that grants access to retrieve objects from a bucket:

index.json Javascript (babel-node)

```
{
   "Version":"2012-10-17",
   "Statement":[
      {
         "Effect":"Allow",
         "Action":[
            "s3:GetObject"
         ],
         "Resource":[
            "arn:aws:s3:::<bucket-name>/*"
         ]
```

Ask

```
}
```

Sample policy for ECS task role

These roles are referenced in the ECS task definition, making permission management modular and repeatable.

Environment variable management

ECS enables us to pass configuration parameters, such as database URLs or API keys, to our containers through environment variables defined in the task definition. These are specified under the `environment` parameter in the `containerDefinitions` section of the task definition. Each environment variable is expressed as a key-value pair, for example:

index.json C++

```
"environment": [
  { "name": "DB_HOST", "value": "mydb.example.com" },
  { "name": "API_KEY", "value": "abcdef123456" }
]
```

Sample environment variables

Ask

This method is especially useful for injecting runtime settings required during container initialization. However, to avoid hardcoding secrets in plain text, we can use the `valueFrom` attribute to reference secure values stored in AWS Systems

Manager Parameter Store or AWS Secrets Manager. This not only enhances security but also centralizes configuration management.

> **Note:** Ensure that either the task execution role or task role, depending on the retrieval method, includes permissions to access the referenced parameters or secrets.

Environment variables configured this way allow teams to easily adapt container behavior across environments without modifying the container image.

The `entryPoint` parameter

In addition to environment variables, ECS allows developers to override the default command defined in a Docker image using the `entryPoint` parameter. This is also configured inside the `containerDefinitions` block of the task definition.

The `entryPoint` parameter is an array of strings that represents the command to run when the container starts, for example:

index.json C++

```
"entryPoint": ["/app/start.sh"]
```

Ask

Sample entry point parameter

This is useful when we need to run a specific script, wrap our application's bootstrapping logic, or customize runtime behavior. It is not used to define environment variables.

> Confusing `entryPoint` with configuration injection is a common mistake. Environment variables must be passed explicitly using the `environment` field.

Amazon ECS provides native integration with Amazon CloudWatch to enable observability of container workloads.

Monitoring in Amazon ECS

Logging and monitoring are essential for debugging, understanding behavior under load, identifying failures early, and triggering alerts for proactive response. To monitor ECS resources, we have the following options:

Tool Name	Type	Description
Amazon CloudWatch Alarms	Automated	Monitor specific metrics over defined time periods and perform actions based on thresholds. This is useful for scaling ECS services and container instances based on CPU and memory utilization.
Amazon CloudWatch Logs	Automated	Collect and store logs from containers by specifying the `awslogs` log driver in task definitions. It also allows monitoring of the operating system and ECS agent logs on container instances.
Amazon CloudWatch Events	Automated	Respond to changes in AWS resources by routing events to target functions or streams, enabling a mated corrective actions.
		Collect, aggregate, and summarize metrics and logs from containerized applications. It provides automated

Ask